Software Engineering Project Proposal

Quran Micro-Learning

Graduation Project 1 - SE3501

Wael Kweder	446017146	Abdulwahid Ghalib	446017073
Abdulrahman Zwobe	446017088	Mohammed Abdulwahab	446016930
Moayed Bassyoni	446002317	Khalid Molfiter	446017087

Supervisor: Dr. Saeed Altalhi

Umm Al-Qura University

October 15th 2026

Table of Contents

1. Introduction 2
1.1 Project Background 2
1.2 Project Purpose 2
1.3 Project Importance 2
2. Problem Statement 2
3. Objectives 3
4. High-Level Requirements Statement 3
4.1 Functional Requirements 3
4.2 Non-Functional Requirements 4
4.3 Constraints 5
5. Project Timeline 5
6. Tools and Technologies 7
7. Expected Outcomes and Deliverables 10
7.1 Software Components (Working Systems) 10
7.2 Technical Documentation 10
7.3 Design & Prototype Artifacts 11
7.4 Presentations & Demonstrations 11
8. References 12

1. Introduction

1.1 Project Background

In tradition, Quran memorization has relied on constant supervision by a teacher and long study sessions. As highly effective as it is, this traditional educational way asks for time commitments and In-person attendance, which is difficult for individuals with demanding academic or professional schedules to adjust to [1]. In response to this lifestyle, the online world has introduced Quran applications and audio-based learning tools that provide learners with more flexible and accessible approaches to Quran memorization [2]. However, existing digital Quran applications vary in their features and approaches to memorization, and many still place responsibility on learners to maintain consistent study routines and organize their learning activities [2], [3]. This shift toward independent self-learning highlights a structural gap between traditional teacher-guided memorization and technology-assisted learning. Therefore, there is a need for an active and structured system that can provide learners with guided learning, monitor their progress, and support them in managing their memorization journey [2], [3].

1.2 Project Purpose

The primary purpose of this project is to develop a micro-self learning mobile application designed for Quran memorization, retention reinforcement and constant tracking across the entire Quran. The application shifts from traditional, slow methods to a structured, roadmap-based way of learning.

1.3 Project Importance

The importance of this project is its ability to make Quran memorization and retention structured and manageable. The application benefits different types of users:

- **Helping Busy Users:** The application benefits busy users by allowing them to utilize short times in their day.
- **Easing Self-Learning:** It provides a comprehensive learning environment for those who face difficulties memorizing without a traditional teacher or can't go to physical memorization circles.
- **Strengthening Retention:** It helps with the issue of forgetting by including a specific Retention Reinforcement section. This allows current learners and established Huffaz to revisit the memorized Surahs through retention-focused exercises, interactive quizzes, and error tracking, ensuring that memorization is lasting.
- **Increasing Motivation:** The application solves lack of motivation through reminder and motivation notifications.

2. Problem Statement

Many Muslims who wish to memorize the Quran face a big challenge. New learners often don't know where or how to begin, have limited communication with teachers, and face a problem finding time to long memorization sessions into busy schedules. Without structure, feedback, or visible progress, they become bored, lose ambition, and often stop [4]. Learners who have

memorized part of the Quran face a different problem: review is time-consuming and frequent, without a structured way to review and consolidate what they know, they gradually forget it [5]. Current digital solutions, such as Quran memorization applications, have introduced features including memorization activities, performance tracking, and recitation assessment [3]. However, there remains an opportunity to integrate structured short lessons, continuous recitation assessment, progress tracking, and organized revision into a single learning platform.

3. Objectives

- **Micro-Learning:** To build a specific roadmap that turns the Quran Surahs into smaller and more manageable chunks to memorize easier.
- **Short-on-Time:** To build specific memorization plans for people with limited time (e.g. 15 min) to help them memorize the Holy Quran with the time limitation.
- **Exercises:** To build engaging and interacting exercises for unmemorized exercises, and dedicate specific exercises for retention to ensure users can retain and memorize effectively.
- **Tracking:** To build a dashboard that tracks user progress with different metrics such as consistency, previous mistakes, and Quran memorization percentage.
- **Reminders:** To send notifications that remind the user of memorization and retention of the Holy Quran Surahs based on the user preferred time to read.

4. High-Level Requirements Statement

4.1 Functional Requirements

The system shall provide:

1. **User Profile and Starting Point**
    - Allow users to create a profile and show their current level of Quran memorization.
    - Personalize the memorization starting point to match the user's existing progress.
2. **Quran Roadmap**
    - Provide a roadmap for each Surah, breaking lessons down verse by verse.
    - Allow users to access memorized verses and continue lessons from where they stopped.
    - Keep future lessons locked until users complete the required learning or review checkpoint.
3. **Memorization Exercises**
    - Design interactive exercises to test recall instead of recognition.
    - Include different types of exercises to help with memorization.
    - Let users recite using voice recognition and receive feedback for each word.
    - Allow typing as an alternative when voice recording is not available.
4. **Checkpoints and Review**
    - Add a review checkpoint after every half-page and at the end of each page.

- Users can only continue after passing the required checkpoint.

5. **Reinforcement Exercises**
   - Add a separate section for Surahs that users have memorized.
   - Allow users to pick a memorized Surahs and do exercises to improve retention.
6. **Progress Tracking**
   - Track which verses each user has memorized and their progress in each Surah.
   - Track daily activity to help users maintain their learning streak.
   - Record users' errors and their past performance.
7. **Motivation and Engagement**
   - Send users motivational notifications and reminders.
   - Break the learning process into short lessons that fit into users' free time.
8. **Study Schedule and Planning**
   - Allow users to set their own schedule by choosing days and times for Quran learning.
   - Let users organize lessons according to their chosen days and times.
   - Notify users of their scheduled lessons through the mobile app.

4.2 Non-Functional Requirements

The system shall maintain:

1. **Usability**
   - Offer a simple, clear, and user-friendly interface for students, young adults, busy users, and self-learners.
   - Make sure users can finish lessons in a short amount of time.
2. **Performance**
   - Ensure quick responses to user actions and smooth navigation between lessons and exercises.
   - Process voice recordings and provide per-word feedback in a timely manner.
3. **Reliability**
   - Reliably save users' progress, completed lessons, streaks, and mistakes.
   - Make sure learning progress is not lost if the app is closed or interrupted.
4. **Accessibility**
   - Offer typing as an option when voice interaction is not available.
   - Help users who have limited time or do not have access to a teacher.
5. **Security and Privacy**
   - Keep users' personal information, learning progress, and voice recordings secure from unauthorized access.
   - Only allow authorized users and system parts to access user data.
6. **Maintainability**
   - Make it easy to add new memorization exercises, review methods, and retention tools without big system changes.

4.3 Constraints

The system shall be developed under the following constraints:

1. **Time Constraint**
   - Lessons should be short enough so users can finish them during brief free periods.
2. **Recall-Based Learning Constraint**
   - Memorization exercises should focus on recall, not just recognition.
3. **Voice Interaction Constraint**
   - Voice-based exercises require voice recording and processing to be available.
   - Typing should be available as an alternative if voice interaction is not possible.
4. **Quran Content Constraint**
   - The system should support the full Quran and organize content by Surah and verse.
5. **Progression Constraint**
   - Upcoming lessons shall remain locked until the required learning or review checkpoint is successfully completed.
6. **Notification Constraint**
   - Send notifications and reminders only through the mobile app.

5. Project Timeline

Milestone	Date (2026)
Project Idea Completed	1 Oct
Chapter 1 Completed	5 Oct
Project Proposal Submitted	8 Oct
Chapter 2 Completed	15 Oct
Model Choice Validated	22 Oct
Chapter 3 Completed	25 Oct
System Design Completed	5 Nov
Interactive Design Prototype Completed	19 Nov
Chapter 4 Completed	3 Dec
Final Report Submitted	10 Dec
Final Presentation Delivered	17 Dec

Table 1 Project milestone list – marked with ◆ in Figure 1

Figure 1 Project timeline - Gantt chart

Phase / Task	Start	End	W1 23 Aug	W2 30 Aug	W3 6 Sep	W4 13 Sep	W5 20 Sep	W6 27 Sep	W7 4 Oct	W8 11 Oct	W9 18 Oct	W10 25 Oct	W11 1 Nov	W12 8 Nov	W13 15 Nov	W14 22 Nov	W15 29 Nov	W16 6 Dec	W17 13 Dec
Phase 1: Project Initiation & Introduction (Chapter 1)	**23 Aug**	**8 Oct**																	
Conduct team meetings to establish background	23 Aug	28 Aug																	
Formulate precise problem statement	29 Aug	3 Sep																	
Define main project aim and measurable objectives	4 Sep	8 Sep																	
Research and review related existing systems	9 Sep	15 Sep																	
Summarize findings into Project Idea and Abstract ◆	16 Sep	1 Oct																	
Draft Chapter 1 for Final Report ◆	20 Sep	5 Oct																	
Complete Project Proposal submission ◆	6 Oct	8 Oct																	
Phase 2: Project Planning Phase (Chapter 2)	**1 Sep**	**15 Oct**																	
Define project scope and context boundaries	1 Sep	10 Sep																	
Identify and document project and product risks	11 Sep	20 Sep																	
Outline software and hardware requirements	21 Sep	30 Sep																	
Create detailed project schedule and Gantt chart	1 Oct	10 Oct																	
Review and compile Chapter 2 sections ◆	11 Oct	15 Oct																	
Phase 3: Technical Feasibility & Model Validation	**9 Sep**	**22 Oct**																	
Research speech recognition models & compare licenses	9 Sep	17 Sep																	
Set up development environment and test models	18 Sep	30 Sep																	
Develop basic word-level comparison prototype	1 Oct	15 Oct																	
Evaluate prototype and validate model choice ◆	16 Oct	22 Oct																	
Phase 4: Requirements Engineering & Analysis (Chapter	**20 Sep**	**25 Oct**																	
Execute requirements collection techniques	20 Sep	30 Sep																	
Define Functional Requirements and list system features	1 Oct	8 Oct																	
Identify Nonfunctional Requirements (quality & constraints)	9 Oct	12 Oct																	
Draw UML Use Case Diagrams for system interactions	13 Oct	18 Oct																	
Document detailed Use Case main flows and alternatives	19 Oct	23 Oct																	
Finalize Chapter 3 compilation ◆	24 Oct	25 Oct																	
Phase 5: System Architecture & Design (Chapter 4 - Part	**15 Oct**	**5 Nov**																	
Draft overall software architecture models	15 Oct	22 Oct																	
Map Use Cases to internal interactions via Sequence Diagrams	23 Oct	28 Oct																	
Design Class Diagrams including attributes and methods	25 Oct	31 Oct																	
Structure Data Storage Organization and DB schema	1 Nov	4 Nov																	
Finalize System Design documentation ◆	4 Nov	5 Nov																	
Phase 6: UI/UX Prototyping & Design Validation (Chapter	**18 Oct**	**3 Dec**																	
Sketch user flows and low-fidelity wireframes	18 Oct	26 Oct																	
Design high-fidelity UI screens (Roadmap, Lesson, Stats)	27 Oct	10 Nov																	
Link screens into a clickable interactive prototype ◆	11 Nov	19 Nov																	
Conduct user feedback sessions for design validation	20 Nov	26 Nov																	
Review prototype with supervisor and gather feedback	22 Nov	27 Nov																	
Refine UI and integrate UI snapshots into Chapter 4 ◆	28 Nov	3 Dec																	
Phase 7: Final Report Compilation	**15 Nov**	**10 Dec**																	
Draft abstract, acknowledgments, and conclusion sections	15 Nov	22 Nov																	
Compile chapters 1-4 into the SWE GP1 Template	23 Nov	28 Nov																	
Format list of tables, figures, acronyms, and references	29 Nov	2 Dec																	
Conduct final proofreading and formatting review	3 Dec	7 Dec																	
Submit Final Report to supervisor ◆	8 Dec	10 Dec																	
Phase 8: Project Presentation & Demo	**1 Dec**	**16 Dec**																	
Outline presentation structure based on report chapters	1 Dec	5 Dec																	
Create presentation poster focusing on design and results	6 Dec	11 Dec																	
Prepare and test live demo/video of the clickable prototype	8 Dec	13 Dec																	
Conduct presentation rehearsal and timing checks	14 Dec	16 Dec																	
Deliver Final Presentation ◆	17 Dec	17 Dec																	
Deadlines	**Date**																		
Project idea	1 Oct							◆											
Proposal	8 Oct								◆										
Final report	10 Dec																	◆	
Presentation	17 Dec																		◆

6. Tools and Technologies

Layer	Category	Tool	Purpose
Frontend	UI framework	Flutter [6]	Single codebase for the Android, iOS, and web interface
	Notifications	flutter_local_notifications [7]	Reminders and personalized motivational messages fetched from the backend and scheduled locally on Android and iOS; in-app reminder banners on web
Backend	Language / runtime	TypeScript [8], Node.js (v24 LTS) [9]	Type-safe backend that runs TypeScript natively, with no separate build step
	Framework	Express.js [10]	REST API routing and middleware.
	Validation	Zod [11]	Validation of request bodies and inputs
	Authentication	jsonwebtoken [12], bcrypt [13]	Secure login, session tokens, and password hashing; secure storage on mobile, httpOnly cookies or short-lived tokens on web
	Email service	Resend [14]	Transactional emails (verification, password reset) and optional reminder emails for web users

Layer	Category	Tool	Purpose
	Phonetic Transcription AI Model	facebook/wav2vec2-xlsr-53-espeak-cv-ft [15]	Extracts phonetic sequences from audio recordings. Supports WebM/Opus and AAC/M4A recordings through audio preprocessing.
	ASR service	Python [16], FastAPI [17]	A cloud-hosted microservice that loads the phonetic transcription model at startup and processes audio recordings to generate phoneme-level predictions for comparison with reference Qur'anic phonemes. Communicates with the Express API to provide recognition results for recitation error detection. |
Data	Database	PostgreSQL [18]	Persistent storage of relational data (users, progress, mistakes)
	ORM	Prisma [19]	Type-safe database access and schema migrations
Infrastructure	Backend, Database & ASR	DigitalOcean [20]	Hosting the Express API, PostgreSQL database, and the ASR service on the same Linux server to minimize latency
	Web hosting	Vercel [21]	Hosting for the Flutter web app over HTTPS (required for browser microphone access)

Layer	Category	Tool	Purpose
	Domain	Namecheap [22]	Custom domain for the API and web app, plus verified email sending
Testing	Automated testing (backend)	Vitest [23], Supertest [24]	Unit tests and API integration tests
	Automated testing (app)	flutter_test [25], integration_test [26]	Widget and integration tests for mobile and web
	API testing	Postman [27]	Manual endpoint testing
	Device testing	Android Emulator [28], iOS Simulator [29], physical devices	Running and demoing the app on both mobile platforms; physical devices used to verify scheduled notification timing
	Browser testing	Chrome [30], Safari [31] (desktop and mobile)	Verifying the web app, especially microphone access
Development	Version control	Git [32], GitHub [33]	Source control and team collaboration
	AI Assistance	Claude Sonnet 5.5 [34], GPT-5.6 Luna [35]	AI-assisted code generation, debugging, and problem solving
	Hardware	Development laptops (8 GB+ RAM), one Mac [36]	Development and local testing; the Mac is required for iOS builds and Safari testing
Design	UI/UX Design	Figma [37]	Designing and prototyping the application interface, user flows, and interactive UI/UX designs

Table 2 Project tools and technologies

7. Expected Outcomes and Deliverables

The main objective of this project is to develop and complete a functional working mobile and web application for learning, reviewing and memorizing the Quran. It is a self-learning system with a micro-structured approach that can be used in a structured and sustainable way for learning, reviewing, and making retention structured, manageable, and sustainable. The project deliverables are categorized into software components, technical documentation, design artifacts, and project presentations.

7.1 Software Components (Working Systems)

1. **Cross-Platform Mobile and Web Application:**
   - **Core Learning Roadmap Module**: Provides users with an Interactive Surah roadmaps that have verse-level granularity (locked, current, completed nodes) and allows users to start at any Surah of their choice.
   - **Interactive Micro-Learning Engine:** A set of short interactive learning exercises (i.e. focused on active recall rather than recognition) which contain audio recitation with word-by-word feedback/validation and a fallback to a keyboard typing mode.
   - **Checkpoint Review Mechanism:** These milestone assessments are enforced after every half page, full page, so that students can reinforce their memorization prior to proceeding to the next part of the content.
   - **Dedicated Retention Reinforcement Section:** for review of learned Surahs by means of retention-oriented exercises
   - **Progress Tracking Dashboard:** An analytics engine to track daily progress (e.g. longest daily streak, memorized percentage, error logs).
   - **Notification & Reminder Subsystem:** Builds consistency into user's daily routine with customizable automated reminders.
2. **Backend API & Database Architecture:**
   - **RESTful API Service:** A Node.js/Express service that provides the various endpoints for authentication, submitting exercises, integrate with speech processing functionality, and user state management.
   - **Relational Database:** A robust PostgreSQL database structured via Prisma ORM to store user profiles, roadmap progress, recitation error history, and system configurations.
   - **Python ASR Microservice:** A cloud-hosted Python service that uses the facebook/wav2vec2-xlsr-53-espeak-cv-ft model to extract phonetic sequences from audio recordings and return recognition results to the backend for comparison with reference Qur'anic phonemes to detect potential recitation errors..

7.2 Technical Documentation

The technical documentation for this project is organized in three main deliverables:

- **Project Proposal:** Prepared at the initial phase to define the project scope, objectives, system requirements, methodology, and execution timeline.

- **Bi-Weekly Progress Logs & Submission:** The project team will submit documented records from their supervisor meeting. These logs detail discussions held, progress reviewed and feedback given in order to maintain alignment with overall project milestones.
- **Final Report:** Comprehensive software design and analysis documentation for Graduation Project covering the scope of the project and its schedule, all functional and non-functional requirements, design of the architecture and all UI prototypes.

7.3 Design & Prototype Artifacts

- **UI/UX Prototype:** High-fidelity interactive prototype for user flows, Surah roadmaps, exercise interface and reinforcement section, designed for mobile.

7.4 Presentations & Demonstrations

- **Final Poster Presentation (Week 16):** Present brief summary of core problem, proposed solution, main features of the prototype and results obtained.
- **User Interface (UI) Prototype Demonstration:** A simple presentation to demonstrate the UI to present key screen flows, main features and user navigation.

8. References

[1] H. Ashar, "Implementation of Talaqqi Method in Memorizing Al-Qur'an at School of Hafidz Qur'an of Muhajirin in Central Gresik," *Stud. Religia J. Pemikir. Dan Pendidik. Islam*, vol. 1, no. 1, 2017, doi: 10.30651/sr.v1i1.1082.

[2] K. Haryono, R. A. Rajagede, and M. U. A. S. Negara, "Quran Memorization Technologies and Methods: Literature Review," *IJID Int. J. Inform. Dev.*, vol. 11, no. 1, pp. 192–201, 2022, doi: 10.14421/ijid.2022.3746.

[3] N. M. Mustafa, Z. Mohd Zaki, K. A. Mohamad, M. Basri, and S. Ariffin, "Development and Alpha Testing of EzHifz Application: Al-Quran Memorization Tool," *Adv. Hum.-Comput. Interact.*, vol. 2021, no. 1, p. 5567001, 2021, doi: 10.1155/2021/5567001.

[4] N. A. Arifi, Z. Zurqoni, and R. Robingatin, "Challenges in Qur'anic Memorization Learning at Elementary Schools: Systematic Literature Review," *Tafkir Interdiscip. J. Islam. Educ.*, vol. 7, no. 1, pp. 188–202, 2026, doi: 10.31538/tijie.v7i1.2481.

[5] N. Yusup, M. M. Abdul Rahim, and A. bin Borham, "MURAJAAH IN QURAN MEMORIZATION AMONG ISLAMIC STUDENTS: A SYSTEMATIC LITERATURE REVIEW," *Int. J. Mod. Educ.*, vol. 7, pp. 17–36, Mar. 2025, doi: 10.35631/IJMOE.724002.

[6] "Flutter - Build apps for any screen." Accessed: Oct. 15, 2026. [Online]. Available: https://flutter.dev/

[7] "flutter_local_notifications | Flutter package," Dart packages. Accessed: Oct. 15, 2026. [Online]. Available: https://pub.dev/packages/flutter_local_notifications

[8] "JavaScript With Syntax For Types." Accessed: Oct. 15, 2026. [Online]. Available: https://www.typescriptlang.org/

[9] "Node.js — Node.js 24.21.0 (LTS)." Accessed: Oct. 15, 2026. [Online]. Available: https://nodejs.org/en/blog/release/v24.21.0

[10] "Express.js · Node.js web application framework," Express.js. Accessed: Oct. 15, 2026. [Online]. Available: https://expressjs.com/

[11] "Intro | Zod." Accessed: Oct. 15, 2026. [Online]. Available: https://zod.dev/

[12] "jsonwebtoken - npm." Accessed: Oct. 15, 2026. [Online]. Available: https://www.npmjs.com/package/jsonwebtoken

[13] "bcrypt," npm. Accessed: Oct. 15, 2026. [Online]. Available: https://www.npmjs.com/package/bcrypt

[14] "Resend · Email for developers." Accessed: Oct. 15, 2026. [Online]. Available: https://resend.com/

[15] “facebook/wav2vec2-xlsr-53-espeak-cv-ft · Hugging Face.” Accessed: Oct. 15, 2026. [Online]. Available: https://huggingface.co/facebook/wav2vec2-xlsr-53-espeak-cv-ft

[16] “Welcome to Python.org,” Python.org. Accessed: Oct. 15, 2026. [Online]. Available: https://www.python.org/

[17] “FastAPI - FastAPI.” Accessed: Oct. 15, 2026. [Online]. Available: https://fastapi.tiangolo.com/

[18] “PostgreSQL: The world’s most advanced open source database.” Accessed: Oct. 15, 2026. [Online]. Available: https://www.postgresql.org/

[19] “Prisma | Agent Infrastructure for TypeScript,” Prisma. Accessed: Oct. 15, 2026. [Online]. Available: https://www.prisma.io

[20] “AI-Native Cloud | DigitalOcean.” Accessed: Oct. 15, 2026. [Online]. Available: https://www.digitalocean.com

[21] “Agentic Infrastructure,” Vercel. Accessed: Oct. 15, 2026. [Online]. Available: https://vercel.com

[22] “Buy a domain name - Register cheap domain names from $0.99 - Namecheap.” Accessed: Oct. 15, 2026. [Online]. Available: https://www.namecheap.com/

[23] “Vitest | Next Generation testing framework.” Accessed: Oct. 15, 2026. [Online]. Available: https://vitest.dev/

[24] “supertest - npm.” Accessed: Oct. 15, 2026. [Online]. Available: https://www.npmjs.com/package/supertest

[25] “flutter_test library - Dart API.” Accessed: Oct. 15, 2026. [Online]. Available: https://api.flutter.dev/flutter/flutter_test/

[26] “Check app functionality with an integration test.” Accessed: Oct. 15, 2026. [Online]. Available: https://docs.flutter.dev/testing/integration-tests

[27] “API engineering for your agents.” Accessed: Oct. 15, 2026. [Online]. Available: https://www.postman.com/

[28] “Download Android Studio & App Tools - Android Developers.” Accessed: Oct. 15, 2026. [Online]. Available: https://developer.android.com/studio

[29] “developer.apple.com/xcode/.” Accessed: Oct. 15, 2026. [Online]. Available: https://developer.apple.com/xcode/

[30] “Google Chrome - The Fast & Secure Web Browser Built to be Yours.” Accessed: Oct. 15, 2026. [Online]. Available: https://www.google.com/chrome/

[31] “Safari - Apple (SA).” Accessed: Oct. 15, 2026. [Online]. Available: https://www.apple.com/sa/safari/

[32] “Git.” Accessed: Oct. 15, 2026. [Online]. Available: https://git-scm.com/

[33] “GitHub.” Accessed: Oct. 15, 2026. [Online]. Available: https://github.com/

[34] “Introducing Claude Sonnet 5.5.” Accessed: Oct. 15, 2026. [Online]. Available: https://www.anthropic.com/claude-sonnet-5-5

[35] “GPT-5.6 Luna Model | OpenAI API,” OpenAI Developers. Accessed: Oct. 15, 2026. [Online]. Available: https://developers.openai.com/api/docs/models/gpt-5.6-luna

[36] “Mac - Apple (SA).” Accessed: Oct. 15, 2026. [Online]. Available: https://www.apple.com/sa/mac/

[37] “Figma: The collaborative canvas for design, code, and AI,” Figma. Accessed: Oct. 15, 2026. [Online]. Available: https://www.figma.com/